简历编写建议

对于社招来说，你可以将其包装到现有的真实项目中，具体的策略可以参考这篇文章：

面试官问“这个项目哪里来的”时，如何回答？

本项目专注 AI 方向，没有涉及到用户认证管理和登录，但你在简历上描述的时候可以补上。下面是一些认证授权相关的工作内容描述：

- 整合 Spring Security + JWT 实现无状态认证，设计双 Token（Access/Refresh）续期机制，权限模型采用 RBAC。
- 基于Sa-Token + Redis 实现了 Token 认证方案，支持 Token自动续签和踢人下线，采用 Bcrypt 算法加密保存密码，权限模型使用 RBAC。
- 采用 Spring Session + Redis 实现集群会话共享，通过 Cookie 和 Header 双通道传递 SessionId，支持记住我功能和并发会话控制（限制单账号 3个终端）。
- 基于 Spring Cloud Gateway + JWT 构建微服务统一认证网关，通过 GlobalFilter 拦截验证，成功后向下游传递用户上下文，实现透明化认证。

相关的面试准备可以参考这篇文章：

面试常见问题补充

后端简历写法

需要注意的是，我这里部分工作内容描述介绍的比较详细，你在描述简历的时候可以适当简化一下。

后端方向版本一（架构与分布式能力侧重）

项目描述:

基于大语言模型的简历分析与模拟面试系统，为求职者提供简历评估建议、个性化面试问题生成和 AI 对话练习服务，支持简历/知识库文档上传、智能问答与 PDF 报告导出等功能。

技术栈:

Java 21 + Spring Boot 4.0 + Spring AI 2.0 + PostgreSQL + pgvector + Redis + Redisson 4.0 + Apache Tika + iText 8 + Docker + Gradle 8.14

工作职责:

- 启用 Java 21 虚拟线程，利用轻量级线程模型大幅提升 I/O 密集型场景（SSE 长连接、AI 调用）的并发吞吐，无需调整线程池配置即可支持千级并发。
- 设计 Redis Stream 模板化异步消费架构，实现简历分析/知识库向量化/面试报告生成的异步解耦，支持自动重试与并发消费，接口响应时间从 15s 降低至 200ms。
- 设计分批评估流水线，通过动态切片（默认 8 题/批）规避 LLM 上下文窗口溢出风险，引入二次总结降级机制确保评估报告生成的鲁棒性。封装 StructuredOutputInvoker 统一处理结构化输出解析失败的重试逻辑，提升 LLM 调用的可靠性。
- 实现基于 PostgreSQL + pgvector 的 RAG 检索增强系统，采用文档分块（TokenTextSplitter）+ 阿里云 text-embedding-v3 向量化（1024 维）+ HNSW 索引存储，满足中小规模知识库的检索性能需求。
- 优化 RAG 检索效果，通过前置过滤（filterExpression）将知识库隔离逻辑下沉至向量库层执行，配合 Query Rewrite + 动态参数矩阵 + 短 Token 物理命中判定，有效拦截 RAG 幻觉输出。

- 构建基于 S3 协议的统一存储抽象层，使用 AWS S3 SDK 兼容 RustFS 对象存储，实现面试附件的统一管理与持久化。
- 封装 Redis + Lua 分布式限流组件，实现滑动窗口算法支持 GLOBAL/IP/USER 多维度令牌桶限流，通过 Hash Tag 优化确保 Redis Cluster 下 Key 落在同一槽位，支持声明式配置 fallback 降级方法。
- 使用 Apache Tika 实现多格式文档解析，结合预编译正则清理控制字符、图片链接等噪声，针对 PDF 禁用图片提取避免临时文件污染，针对 DOCX 禁用嵌入资源提取减少噪声。
- 基于 iText 8 实现 PDF 报告导出，内置珠圆玉润仿宋字体支持跨平台渲染，实现分数配色与多维度排版设计。
- 编写 Docker Compose 配置文件实现一键部署，通过环境变量管理敏感配置（如 AI API Key），确保开发、测试与生产环境的一致性。

后端方向版本二（AI 应用与流式输出侧重）

项目描述:

集成大语言模型能力的智能面试辅助平台，提供简历 AI 分析、模拟面试问答、知识库 RAG 检索三大核心功能，支持多文档上传、流式对话与报告导出。

技术栈:

Java 21 + Spring Boot 4.0 + Spring AI 2.0 + PostgreSQL + pgvector + Redis + Reactor + Apache Tika + 阿里 DashScope + MapStruct

工作职责:

- 深度集成 Spring AI 2.0 框架，封装 StructuredOutputInvoker 统一处理结构化输出解析失败的重试逻辑，支持配置化重试次数与错误注入，提升 LLM 调用的可靠性。
- 实现基于 PostgreSQL + pgvector 的 RAG 检索增强系统，采用文档分块（TokenTextSplitter）+ 阿里云 text-embedding-v3 向量化（1024 维）+ HNSW 索引存储，满足中小规模知识库的检索性能需求。
- 设计基于 Query Rewrite 的 RAG 增强架构，通过 LLM 将短查询、模糊语义转化为可检索语句，配合短 Token 物理命中判定，将语义检索准确率提升 35% 以上。
- 构建 RAG 会话持久化管理方案，采用"预创建占位 + 异步流式更新"策略，确保存储层能够实时记录 AI 逐字生成的完整对话内容。
- 设计多级模拟面试引擎，支持主问题与追问（Follow-up）的线性问答流，通过持久化会话状态机确保面试体验的连贯性与深度。
- 基于 SSE 实现流式响应，设计探测窗口算法（120 字符缓冲）快速识别"无信息"模板并归一化拒答话术，避免长篇无效输出，同时保持打字机效果的用户体验。
- 基于 RISEN 框架设计 AI 评估提示词，通过资深专家角色设定、量化评分准则与否定约束，确保评估结果的专业性与一致性。
- 实现问题历史去重机制，将已回答问题（限制 30 道）作为上下文传入 AI，自动生成不重复的新问题，提升面试体验。
- 使用 MapStruct 实现领域实体与 DTO 的编译期自动映射，性能优于反射方案，结合 Lombok 配置增强复杂对象转换的类型安全性。
- 实现基于文件内容特征 SHA-256 哈希的去重检测逻辑，在解析前拦截重复文档上传，有效避免 AI 接口的冗余调用与存储资源浪费。
- 实现向量化分批处理策略，针对阿里云 DashScope API 限制（batch_size ≤ 10），自动分批调用，向量化吞吐量达 500+ 文档/分钟。

后端方向版本三（工程化与基础设施侧重）

项目描述:

基于 Gradle 多模块构建的智能面试平台，实现简历分析、模拟面试、知识库管理等核心功能，支持多文档上传、向量检索与报告导出。

技术栈:

Java 21 + Spring Boot 4.0 + PostgreSQL + pgvector + Redis + Redisson 4.0 + RustFS + Apache Tika + iText 8 + Docker + Gradle 8.14

工作职责:

- 实现 Spring Boot 4.0 迁移，处理 Jackson 序列化重构与 jakarta 包名迁移，实现全局统一响应结构与异常捕获体系的标准化。
- 基于 Redisson 封装 Redis 工程化库，提供键值、Hash、分布式锁、Stream 消息队列、原子计数器、列表等多种操作模式，覆盖缓存、队列、锁三大场景。
- 基于 RISEN 框架（Role/Instructions/Standards/Examples/Negations）设计 AI 评估提示词，通过资深专家角色设定、量化评分准则与否定约束，确保评估结果的专业性与一致性。
- 实现 Redis Stream 阻塞读取优化，使用 BLOCK 参数让服务端等待消息而非客户端轮询，配合 BATCH_SIZE=10 支持高吞吐消费。
- 封装 AbstractStreamConsumer/Producer 模板类收敛消费循环、ACK、重试与生命周期管理逻辑，通过 @PostConstruct 自动初始化消费者组，结合 @PreDestroy 与 AtomicBoolean 实现优雅停机。
- 建立 RAG 动态检索调优矩阵，根据 Query 长度自动切换 Top-K 与相似度阈值（minScore），配合过滤前置策略与短 Token 物理命中判定，有效拦截幻觉输出。
- 实现异步任务状态流转机制（PENDING → PROCESSING → COMPLETED），针对分批评估失败任务提供自动重试与错误降级入口，保障系统在高延迟 AI 调用环境下的可靠性。
- 设计文档分块与 metadata 增强策略，使用 TokenTextSplitter 实现 token-aware 分块，通过 kb_id 前置过滤确保检索准确性。
- 实现文件类型安全校验机制，基于 MIME 类型与文件扩展名双重验证，有效拦截非法文件上传，保障系统文件安全。
- 设计数据库表结构，简历表使用 fileHash 唯一索引实现去重，知识库表通过 vectorStatus 独立管理向量化进度，支持状态追踪。
- 编写 Docker Compose 编排方案，通过环境变量管理敏感配置，支持全套服务的一键部署与配置隔离。

测试/测开简历写法

测试/测开方向版本一（单元测试与测试驱动开发）

项目描述:

智能面试辅助平台的测试体系建设，覆盖文档解析、文本清洗、向量检索、限流组件等核心功能，确保系统质量与稳定性。

技术栈:

JUnit 5 + Mockito + Spring Boot Test + Redis + AssertJ + @TempDir + ArgumentCaptor

工作职责:

- 负责智能面试平台的单元测试体系建设，使用 JUnit 5 + Mockito 模拟依赖组件，设计测试用例覆盖文档解析、文本清洗、向量化等核心逻辑。
- 采用 JUnit 5 现代特性组织测试代码，使用 @Nested 嵌套测试类按功能分组，@DisplayName 提供清晰的中文测试描述，提升测试报告可读性。
- 使用 @ParameterizedTest + @NullAndEmptySource + @ValueSource 实现参数化测试，单个测试方法覆盖空值、边界值、特殊字符等多种场景，减少代码重复。

- 针对 TextCleaningService 编写专项测试，覆盖控制字符清理、图片文件名过滤、URL 清理、分隔线去除、空行压缩等边界场景，确保 RAG 语义输入准确性。
- 设计限流组件集成测试方案，使用 ClassPathResource 加载 Lua 脚本，验证令牌桶算法的多维度限流逻辑，通过 StringCodec 确保与 Redis 的编码兼容性。
- 使用 Mockito ArgumentCaptor 实现复杂业务逻辑的深度验证，通过 captor.getAllValues() 捕获多次调用参数，验证向量化分批处理每批不超过 10 个文档。
- 实现解析服务的集成测试，使用 @TempDir 管理临时文件，支持真实 PDF/DOCX/TXT 文件的端到端解析验证，测试异常文件的容错性。
- 利用 AssertJ 的软断言机制优化复杂校验逻辑，在一次测试运行中捕获多个字段的校验失败信息，提升缺陷排查效率。

测试/测开方向版本二（功能测试与异常场景覆盖）

项目描述:

智能面试辅助平台的质量保障工作，覆盖向量检索服务、SSE 流式输出、全局异常处理等核心功能，确保系统在各种异常场景下的稳定性。

技术栈:

JUnit 5 + Mockito + Spring Boot Test + Redis + StepVerifier + AssertJ + inOrder 验证

工作职责:

- 编写向量化服务异常场景测试用例，模拟大模型服务超时、网络抖动及存储异常等情况，验证系统的降级与重试策略。
- 设计向量搜索向后兼容性测试，验证 String 和 Long 两种 kb_id 格式的正确过滤，确保旧数据迁移后的查询兼容性。
- 使用 Mockito inOrder 验证方法调用顺序，确保向量化前先删除旧数据再添加新数据，保证数据一致性。
- 利用 StepVerifier 对响应式流进行测试，验证流式输出的背压控制与错误传播机制，确保 SSE 接口在高并发下的稳定性。
- 设计 Redis Stream 消息队列端到端测试，验证任务从发送、消费到 ACK 确认的全链路闭环，确保异步任务系统具备高可靠性。
- 针对全局异常处理器编写校验用例，验证错误码的准确转换，确保 AI 服务异常（超时、401、429）的正确分类。
- 编写限流注解元配置测试，验证 @Retention、@Target 等注解属性，确保 @RateLimit 注解的默认值和自定义值正确生效。
- 实现文件上传安全测试，覆盖超大文件、非法格式、特殊字符、多语言混合等场景，确保后端安全校验逻辑严密。
- 建立多层级测试数据管理，在 test/resources/test-files 目录组织 TXT/MD 等格式的真实简历样本，支持集成测试的可复现性。

测试/测开方向版本三（AI 与向量化测试专项）

项目描述:

智能面试平台 AI 功能的测试保障工作，覆盖 Spring AI 集成、向量化存储、相似度搜索等核心功能，确保 AI 服务的准确性与稳定性。

技术栈:

JUnit 5 + Mockito + Spring AI Test + pgvector + ArgumentCaptor + @Disabled

工作职责:

- 设计向量化服务完整测试套件，覆盖文本分块、metadata 设置、分批处理、空内容处理等场景，验证 TokenTextSplitter 的分块策略正确性。
- 使用 ArgumentCaptor 捕获 VectorStore.add() 的所有调用参数，验证每批次文档数量不超过 API 限制（10 个），确保分批逻辑正确。
- 编写向量搜索过滤测试，验证基于 knowledgeBaseIds 的 metadata 过滤逻辑，覆盖 String/Long 类型 kb_id 的向后兼容性处理。
- 实现 topK 边界测试，验证 topK 为 0、topK 大于实际结果数等边界场景的正确处理，确保搜索结果的准确性。
- 设计无效 kb_id 格式的异常处理测试，验证非数字格式、null 值等异常数据的正确过滤，保证系统的健壮性。
- 使用 @Disabled 注解标记需要 Redis 服务的集成测试，支持在无外部依赖环境下运行单元测试，便于 CI/CD 流水线集成。
- 编写自定义异常单元测试，验证 RateLimitExceededException 的错误码（8001）、消息继承、异常链传递等行为，确保异常体系的完整性。
- 实现测试数据生成器，封装 generateLongContent()、createMockDocuments() 等工具方法，生成足够长的文本确保 TokenTextSplitter 产生 chunks。

前端简历写法

前端方向版本一（React 性能优化侧重）

项目描述:

基于 React 18 的智能面试辅助平台前端，实现简历上传分析、AI 模拟面试、知识库 RAG 问答等核心功能，支持流式对话、虚拟列表渲染与 Markdown 实时预览。

技术栈:

React 18.3 + TypeScript 5.6 + Vite 5.4 + Tailwind CSS 4.1 + react-virtuoso + react-markdown + Axios + pnpm

工作职责:

- 使用 React 18 + TypeScript 构建智能面试平台前端，采用 Hooks 组件化开发，实现简历分析、模拟面试、知识库问答三大核心功能模块。
- 基于 react-virtuoso 实现 RAG 聊天消息的虚拟列表渲染，解决长对话场景下 DOM 节点过多导致的性能问题。
- 实现基于 Fetch API + ReadableStream 的 SSE 流式消息处理，支持 AI 回复的逐字渲染效果，配合事件缓冲机制处理消息边界。
- 集成 react-markdown + remark-gfm 实现 AI 回复的 Markdown 实时渲染，支持 GFM 语法、代码高亮与表格等富文本展示。
- 使用 Tailwind CSS 4.1 构建响应式 UI 组件库，结合 @tailwindcss/typography 实现一致的排版系统，提升开发效率与视觉一致性。
- 基于 framer-motion 实现页面切换与组件交互动画，提供流畅的用户体验与视觉反馈。
- 封装 Axios 请求拦截器实现统一错误处理与 Loading 状态管理，配合后端错误码体系提供友好的用户提示。
- 使用 Vite 5.4 配置开发与生产环境，实现模块热更新与按需加载。

前端方向版本二（状态管理与用户体验侧重）

项目描述:

智能面试辅助平台的 Web 端应用，提供简历智能分析、AI 对话练习、知识库检索问答等功能，支持多文档管理、实时流式交互与数据可视化。

技术栈:

React 18 + TypeScript + Vite + Tailwind CSS + recharts + react-router-dom 7 + lucide-react + Axios

工作职责:

- 设计并实现前端路由架构，使用 react-router-dom 7 管理多页面导航，支持简历历史、面试记录、知识库管理等模块的无缝切换。
- 基于 recharts 实现面试评分趋势图与简历分析雷达图，直观展示用户的能力分布与进步曲线。
- 实现文件上传组件，支持拖拽上传、格式校验与上传进度展示，配合后端实现 PDF/DOCX/MD 等多格式简历解析。
- 设计知识库会话管理界面，支持多知识库关联、对话历史持久化、会话置顶与删除等功能，提升 RAG 问答的使用体验。
- 封装通用 UI 组件（Button、Modal、Toast、Skeleton），基于 Tailwind CSS 实现主题一致性，降低跨页面开发成本。
- 使用 lucide-react 图标库统一视觉风格，结合 CSS 动画实现加载态与交互反馈，提升界面的专业度。
- 实现面试轮询机制，定时获取异步分析任务状态，配合骨架屏与进度条提供流畅的等待体验。
- 配置 TypeScript 严格模式与 ESLint 规则，确保代码类型安全与团队协作规范。

全栈简历写法

全栈方向版本一（前后端一体化能力）

项目描述:

全栈开发的智能面试辅助平台，前端基于 React 18 + TypeScript，后端基于 Spring Boot 4.0 + Spring AI，实现简历分析、模拟面试、知识库 RAG 问答的完整闭环。

技术栈:

React 18 + TypeScript + Vite + Tailwind CSS | Spring Boot 4.0 + Spring AI 2.0 + PostgreSQL + pgvector + Redis

工作职责:

- 独立完成智能面试平台的全栈开发，前端使用 React 18 + TypeScript 构建 SPA 应用，后端基于 Spring Boot 4.0 提供 RESTful API。
- 前端实现 SSE 流式消息处理与虚拟列表优化，后端基于 Reactor 对接大模型 Streaming API，返回 Flux，打通 AI 对话的完整链路。
- 设计并实现 RAG 知识库功能，前端提供文档上传与会话管理界面，后端集成 pgvector 实现向量检索与语义问答。
- 构建 Redis Stream 异步任务体系，前端通过轮询获取任务状态，后端实现生产者-消费者模式处理耗时的 AI 分析任务。
- 前端封装 Axios 统一请求层，后端设计分层错误码体系，实现前后端错误处理的一致性与可追溯性。
- 实现文件上传全链路，前端支持拖拽与格式校验，后端基于 Apache Tika 解析多格式文档并存储至 S3 兼容存储。
- 使用 Docker Compose 编排前后端服务与中间件，实现开发、测试、生产环境的一键部署与配置隔离。
- 编写后端单元测试与集成测试，使用 JUnit 5 + Mockito，确保核心功能的稳定性。

简历亮点补充建议

可量化的业绩指标

在描述工作职责时，尽量加入量化数据，让面试官对你的贡献有直观感知：

- 接口响应时间从 Xs 降低至 Yms（如：从 15s 降低至 200ms）
- 渲染性能提升 X%（如：虚拟列表优化后性能提升 80%）
- 代码覆盖率达到 X%（如：核心模块单测覆盖率达 85%）
- 并发支持从 X 提升至 Y（如：支持 1000 QPS 的并发请求）

可深挖的技术难点

以下是一些面试官可能深入追问的技术点，建议提前准备：

1. **Redis Stream vs 传统消息队列**：为什么选择 Redis Stream 而不是 RabbitMQ/Kafka？适用场景（轻量级、低延迟）、持久化机制、Consumer Group 的 ACK 确认与 Pending 列表、消费者优雅停机（`@PreDestroy` + `AtomicBoolean`）。
2. **分布式限流的实现细节**：令牌桶 vs 滑动窗口、Lua 脚本原子性、多维度组合限流的两阶段执行（预检查 + 扣减）、Hash Tag 确保 Redis Cluster 槽位一致性、脚本 SHA1 预加载优化、过期令牌自动回收机制。
3. **RAG 检索优化**：向量维度选择（1024 维 vs 768 维）、HNSW 索引原理、相似度算法对比（余弦/欧氏/点积）、Top-K 召回与 metadata 过滤。
4. **Java 21 虚拟线程（Virtual Threads）**：为什么启用虚拟线程？平台线程 vs 虚拟线程的资源消耗对比、适合 I/O 密集型场景（SSE 长连接、AI 调用）、`spring.threads.virtual.enabled` 配置原理、无需调整线程池配置即可支持千级并发、虚拟线程的阻塞语义与 pinning 问题。
5. **SSE vs WebSocket**：为什么选择 SSE？单向 vs 双向通信、浏览器兼容性、自动重连机制、`Flux<String>` 流式返回。流式探测窗口算法（Probe Window）实现原理：缓冲 120 字符识别无信息模板、命中则立即输出固定模板并结束、非命中则尽快释放缓冲继续实时透传。
6. **Spring Boot 4.0 升级挑战**：Jackson 序列化重构（注解变更）、jakarta 包名迁移、Redisson 4.0 适配。
7. **Spring AI 工程化**：`ChatClient` 建造者模式、`PromptTemplate` 动态变量填充、`BeanOutputConverter` 结构化输出、中间 DTO 解耦 AI 响应。
8. **向量化分批策略**：为什么限制 `batch_size` ≤ 10？API 限制处理、分批调用的错误处理与重试。
9. **文档解析容错**：Apache Tika 的 `BodyContentHandler` 限制（5MB）、`NoOpEmbeddedDocumentExtractor` 禁用嵌入资源、PDF 图片提取禁用。
10. **消费者生命周期管理**：`@PostConstruct` 初始化消费者组、`@PreDestroy` 优雅关闭、`AtomicBoolean` 标志位的线程安全控制。

项目难点与解决方案模板

面试中经常会问"项目中遇到的最大挑战是什么"，可以参考以下模板组织回答：

模板：STAR 法则

- **Situation（背景）**：项目中遇到了什么问题？
- **Task（任务）**：你的职责是什么？
- **Action（行动）**：你采取了什么措施？
- **Result（结果）**：最终效果如何？

示例一：简历分析接口响应超时

- **背景**：简历上传后需调用大模型进行分析，单次分析耗时 10-15 秒，同步处理导致接口超时，用户体验极差。
- **任务**：优化简历分析流程，将接口响应时间控制在可接受范围内，同时保证分析任务的可靠执行。
- **行动**：设计基于 Redis Stream 的异步任务队列，接口接收请求后立即返回任务 ID（状态为 PENDING），后台消费者异步执行分析任务，前端通过轮询获取最新状态。同时实现 Consumer Group 支持多实例水平扩展，配合 ACK 确认与自动重试机制保证任务不丢失。

- **结果**：接口响应时间从 15s 降低至 200ms，系统支持多实例部署后吞吐量提升 3 倍，任务失败率从 5% 降低至 0.1% 以下。

示例二：AI 接口被恶意刷量

- **背景**：上线初期发现部分用户/IP 频繁调用 AI 分析接口，导致大模型 API 配额快速耗尽，影响正常用户使用。
- **任务**：设计多维度限流方案，在不影响正常用户体验的前提下，有效拦截恶意请求。
- **行动**：基于 Redis + Lua 脚本封装分布式限流组件，实现 GLOBAL（全局）、IP、USER 三个维度的令牌桶限流。通过自定义 @RateLimit 注解声明式配置限流规则，Lua 脚本保证多维度检查的原子性，使用 Hash Tag 兼容 Redis Cluster 部署。
- **结果**：恶意请求拦截率达 99%，API 配额消耗降低 60%，正常用户请求通过率保持在 99.5% 以上，组件已沉淀为团队通用基础设施。

示例三：多格式文档解析稳定性问题

- **背景**：用户上传的简历格式多样（PDF/DOCX/DOC/TXT），部分文档解析失败或提取内容包含大量乱码，影响后续 AI 分析质量。
- **任务**：构建稳定可靠的多格式文档解析服务，提升解析成功率与内容质量。
- **行动**：引入 Apache Tika 统一处理多格式文档，针对 PDF 禁用图片提取避免临时文件污染，针对 DOCX 禁用嵌入资源提取减少噪声。同时封装 TextCleaningService 进行正则清洗，移除控制字符、标准化空行、清理冗余空白。对于解析失败的文档，实现异常降级策略返回友好提示。
- **结果**：文档解析成功率从 85% 提升至 99%，AI 分析输入文本质量显著提升，相关模块单测覆盖率达 90%。

其他可补充的技术点

如果简历篇幅允许，还可以补充以下内容：

后端方向：

- **Java 21 新特性应用**：虚拟线程（Virtual Threads）在高并发场景的应用（`spring.threads.virtual.enabled=true`）、Record 类简化 DTO 定义、模式匹配增强代码可读性、switch 表达式增强。
- **Spring AI 2.0 深度集成**：ChatClient 声明式 API、PromptTemplate 模板管理、BeanOutputConverter 结构化输出、向量存储抽象层、中间 DTO 解耦 AI 响应。
- **SSE 流式输出优化**：基于 `Flux<String>` 的流式响应、探测窗口算法（120 字符缓冲）识别无信息模板、自动归一化拒答话术、避免长篇无效输出。
- **RAG 检索增强**：Query Rewrite（检索改写）将短查询、模糊语义转化为可检索语句、动态检索参数矩阵（短/中/长 Query 匹配不同 topK 与 minScore）、短 Token 物理命中判定、HNSW 索引原理。
- **文本清理服务**：预编译正则表达式清理控制字符、图片链接、分隔线等、针对 PDF 禁用图片提取避免临时文件污染、针对 DOCX 禁用嵌入资源提取减少噪声。
- **限流组件工程化**：Lua 脚本 SHA1 预加载优化、过期令牌自动回收、声明式配置 fallback 降级方法、X-Forwarded-For 头处理获取真实 IP。
- **RISEN 提示词工程框架**：Role（角色设定）、Instructions（指令描述）、Standards（评估标准）、Examples（示例参考）、Negations（否定约束）。
- **智能问题分布算法**：基于简历内容的加权问题类型分布、历史问题去重、8 大领域覆盖（项目/MySQL/Redis/Java 基础/集合/并发/Spring）。
- **文档解析容错处理**：Apache Tika 的配置优化（禁用嵌入资源提取）、损坏文件的异常降级策略、错误信息截断（500 字符）防止数据库溢出。
- **PDF 导出跨平台一致性**：iText 8 中文字体内嵌方案（珠圆玉润仿宋）、分数配色与排版设计。
- **文件安全校验**：基于 MIME 类型和文件扩展名的双重验证、防止路径穿越攻击、内容类型白名单。
- **数据库索引优化**：fileHash 唯一索引加速去重、category 索引加速分类查询、HNSW 向量索引。

测试方向：

- **JUnit 5 高级特性**：@Nested 嵌套测试类组织、@ParameterizedTest 参数化测试、@DisplayName 中文测试描述、@Tag 测试分类。
- **Mockito 深度应用**：ArgumentCaptor 捕获多次调用参数、inOrder 顺序验证、when-thenAnswer 复杂返回逻辑、doThrow 异常注入。
- **测试数据管理**：@TempDir 临时文件管理、测试资源文件组织（test-files 目录）、测试数据生成器封装。
- **集成测试设计**：@Disabled 标记外部依赖测试、真实文件端到端测试、Redis 集成测试与数据清理。
- **向后兼容性测试**：String/Long 类型 kb_id 的过滤兼容、无效格式的异常处理、数据迁移场景验证。